МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ № 2

на тему «Функції вищого порядку, lambda, композиція»

Виконала:
Студентка 2 курсу
групи ФЕП-22
Сулятицька Христина

Перевірив:
Аспірант
Остріков Данило Андрійович

Львів-2026

Мета. ознайомитися з функціями вищого порядку, lambda-виразами та композицією функцій; навчитися використовувати map, filter і reduce або їх генераторні еквіваленти; реалізувати функціональний конвеєр обробки банківських рахунків без мутації вхідних даних; виконати сортування та формування Top-N рейтингу; перевірити правильність роботи програми за допомогою pytest.

Теоретичні відомості.

Функція вищого порядку — це функція, яка приймає іншу функцію як аргумент або повертає функцію як результат. Такий підхід дозволяє передавати правила обробки окремо від основного алгоритму.

Lambda — це короткий спосіб створення анонімної функції. Lambda-вираз має вигляд lambda аргументи: вираз і використовується для компактного опису простих операцій, наприклад критерію сортування або умови фільтрації.

Функції filter, map і reduce застосовуються для функціональної обробки колекцій. filter відбирає елементи за умовою, map перетворює кожен елемент, а reduce послідовно об'єднує елементи в один підсумковий результат.

Композиція функцій — це об'єднання кількох функцій у єдиний конвеєр, у якому результат однієї операції передається наступній. У роботі для цього реалізовано compose, а також допоміжну функцію pipe.

У функціональному стилі важливо не змінювати початкові структури даних. Для оновлення записів створюються нові словники за допомогою конструкції {**record, ...}. Введення-виведення винесено до main.py, а логіку обробки — до lab2.py.

Для 8-го варіанта обробляються банківські рахунки з полями id, name, city і transactions. Оборот обчислюється як сума транзакцій; для вибраного міста застосовується коефіцієнт, після чого записи фільтруються, сортуються та відбираються до Top-N. За допомогою reduce обчислюються count, turnover та avg_turnover.

Хід роботи

1. Створено структуру проєкту лабораторної роботи, у якій окремо розміщено функціональне ядро, CLI-оболонку, приклад JSON-даних і автоматичні тести:

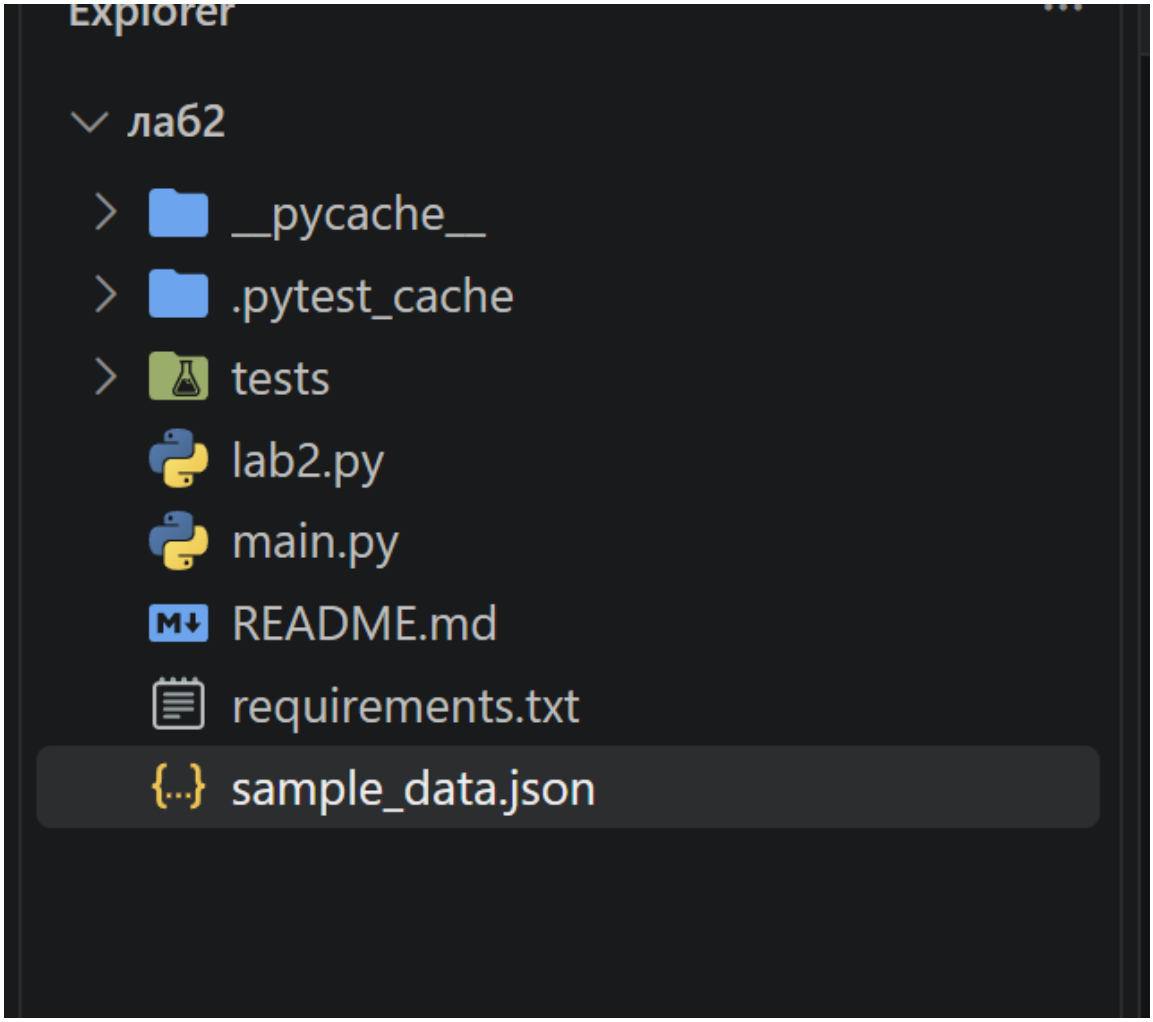


Рисунок 1 — Структура проєкту лабораторної роботи № 2.

2. Реалізовано функціональне ядро програми у файлі lab2.py. У ньому описано типізовані структури Account і Transaction, а також функції композиції та послідовного виконання. Основні перетворення створюють нові записи, не змінюючи початкові словники.



```python
from __future__ import annotations

from functools import reduce
from itertools import islice
from typing import Callable, Iterable, Iterator, TypedDict


class Transaction(TypedDict, total=False):
    amount: float
    type: str


class Account(TypedDict, total=False):
    id: int
    name: str
    city: str
    transactions: list[float] | list[Transaction]
    turnover: float


def compose(*functions: Callable):
    """Композиція: compose(f, g)(x) виконує спочатку g, потім f."""
    def composed(value):
        for function in reversed(functions):
            value = function(value)
        return value
    return composed


def pipe(value, *functions):
    """Послідовно передає результат кожної функції наступній."""
    for function in functions:
        value = function(value)
    return value


def normalize_names() -> Callable[[Iterable[Account]], Iterator[Account]]:
    """Створює нові записи з іменами у форматі Title Case."""
    return lambda accounts: (
        {**account, "name": str(account.get("name", "")).strip().title()}
        for account in accounts
    )


def with_turnover() -> Callable[[Iterable[Account]], Iterator[Account]]:
    """Обчислює оборот рахунку як суму всіх транзакцій."""
    def get_amount(transaction) -> float:
        if isinstance(transaction, dict):
            return float(transaction.get("amount", 0.0))
        return float(transaction)

    return lambda accounts: (
        {
            **account,
            "turnover": sum(
                get_amount(transaction)
                for transaction in (account.get("transactions", []) or [])
            ),
        }
        for account in accounts
    )


def boost_city(city: str, factor: float) -> Callable[[Iterable[Account]], Iterator[Account]]:
    """Застосовує коефіцієнт до обороту рахунків у вибраному місті."""
    def transform(accounts: Iterable[Account]) -> Iterator[Account]:
        for account in accounts:
            turnover = float(account.get("turnover", 0.0))
            new_turnover = turnover * factor if account.get("city") == city else turnover
            yield {**account, "turnover": new_turnover}
    return transform
```

```python
    return transform


def filter_min_turnover(minimum: float) -> Callable[[Iterable[Account]], Iterator[Account]]:
    """Залишає рахунки з оборотом не меншим за minimum."""
    return lambda accounts: (
        account for account in accounts
        if float(account.get("turnover", 0.0)) >= minimum
    )


def sort_by_turnover_desc() -> Callable[[Iterable[Account]], list[Account]]:
    """Сортує за оборотом за спаданням, а за однакового обороту — за id."""
    return lambda accounts: sorted(
        accounts,
        key=lambda account: (
            -float(account.get("turnover", 0.0)),
            int(account.get("id", 0)),
        ),
    )


def take(top_n: int) -> Callable[[Iterable[Account]], list[Account]]:
    """Повертає не більше top_n перших записів."""
    if top_n < 0:
        raise ValueError("top_n не може бути від'ємним")
    return lambda accounts: list(islice(accounts, top_n))


def reduce_stats(accounts: Iterable[Account]) -> dict[str, float]:
    """Обчислює count, turnover й avg_turnover через reduce."""
    initial = {"count": 0.0, "turnover": 0.0}
    result = reduce(
        lambda acc, account: {
            "count": acc["count"] + 1,
            "turnover": acc["turnover"] + float(account.get("turnover", 0.0)),
        },
        accounts,
        initial,
    )
    average = result["turnover"] / result["count"] if result["count"] else 0.0
    return {
        "count": result["count"],
        "turnover": result["turnover"],
        "avg_turnover": average,
    }


def build_pipeline(
    top_n: int = 3,
    city: str = "Kyiv",
    factor: float = 1.1,
    min_turnover: float = 0.0,
):
    """Будує конвеєр обробки банківських рахунків."""
    return compose(
        take(top_n),
        sort_by_turnover_desc(),
        filter_min_turnover(min_turnover),
        boost_city(city, factor),
        with_turnover(),
        normalize_names(),
    )


def process_accounts(
    accounts: list[Account],
    top_n: int = 3,
    city: str = "Kyiv",
    factor: float = 1.1,
    min_turnover: float = 0.0,
) -> list[Account]:
    """Запускає конвеєр й повертає Top-N рахунків."""
    return build_pipeline(top_n, city, factor, min_turnover)(accounts)
```

Рисунок 2 — Опис типів даних і реалізація композиції функцій.

Функція with_turnover обчислює оборот рахунку як суму значень у списку transactions. Функція boost_city застосовує заданий коефіцієнт лише до рахунків обраного міста. Функція filter_min_turnover залишає записи, що відповідають мінімальному порогу. Усі ці операції повертають нові словники або ітератори нових записів, не змінюючи вхідний список.

3. Реалізовано сортування, формування Top-N, статистичну агрегацію та побудову конвеєра. Сортування виконується за оборотом за спаданням, а за однакових значень використовується додатковий ключ id за зростанням. Функція reduce_stats обчислює кількість рахунків, сумарний і середній оборот.

Функція build_pipeline об'єднує етапи обробки за допомогою compose. У конвеєрі спочатку нормалізуються імена, обчислюється оборот, застосовується коефіцієнт міста, виконується фільтрація, сортування та вибір перших N записів. Функція process_accounts забезпечує запуск сформованого конвеєра з переданими параметрами.

4. Винесено введення та виведення даних у файл main.py. Програма зчитує записи з JSON-файлу, приймає параметри командного рядка та виводить сформований рейтинг і статистику.

lab2.py main.py

main.py > main

```python
"""CLI-демонстрація лабораторної роботи №2, варіант 8."""
import argparse
import json
from pathlib import Path

from lab2 import process_accounts, reduce_stats


def main():
    parser = argparse.ArgumentParser(description="Аналіз банківських рахунків")
    parser.add_argument("--data", default="sample_data.json", help="Шлях до JSON-файлу")
    parser.add_argument("--top", type=int, default=3, help="Кількість записів у рейтингу")
    parser.add_argument("--boost-city", default="Kyiv", help="Місто для застосування коефіцієнта")
    parser.add_argument("--factor", type=float, default=1.1, help="Коефіцієнт для міста")
    parser.add_argument("--min-turnover", type=float, default=0.0, help="Мінімальний оборот")
    args = parser.parse_args()

    with Path(args.data).open("r", encoding="utf-8") as file:
        accounts = json.load(file)

    results = process_accounts(
        accounts,
        top_n=args.top,
        city=args.boost_city,
        factor=args.factor,
        min_turnover=args.min_turnover,
    )

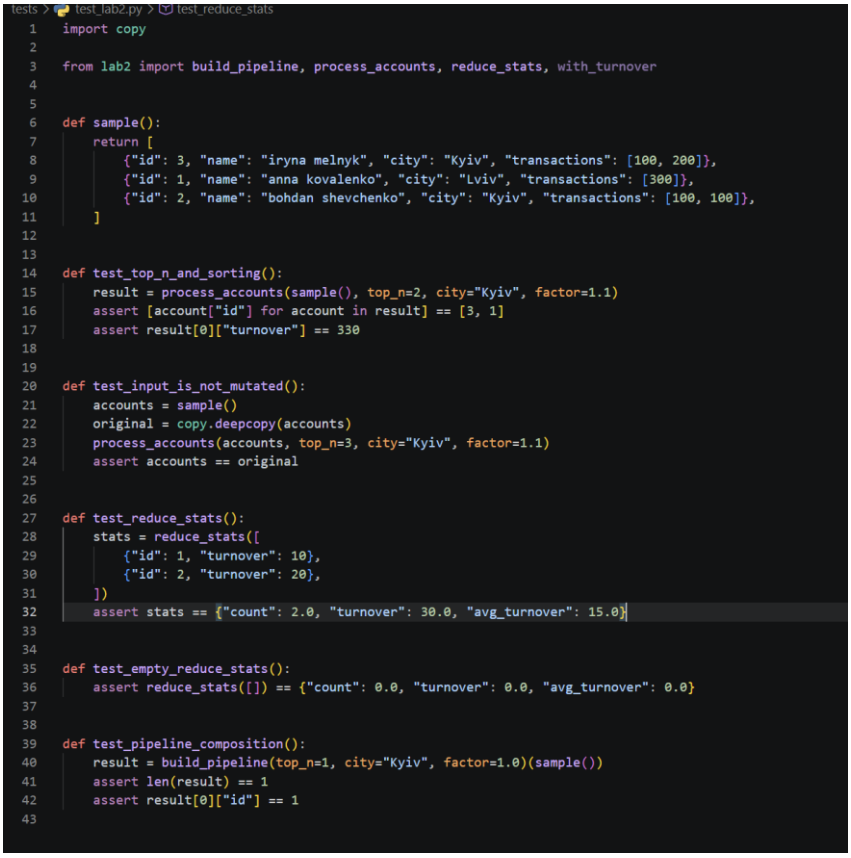
    print("\nТОП рахунків:")
    if not results:
        print("Немає рахунків, які відповідають умовам.")
    for account in results:
        print(
            f"ID: {account.get('id')} | "
            f"Ім'я: {account.get('name')} | "
            f"Місто: {account.get('city')} | "
            f"Оборот: {account.get('turnover', 0):.2f}"
        )

    print("\nСтатистика для показаного рейтингу:")
    stats = reduce_stats(results)
    print(f"Кількість рахунків: {int(stats['count'])}")
    print(f"Загальний оборот: {stats['turnover']:.2f}")
    print(f"Середній оборот: {stats['avg_turnover']:.2f}")


if __name__ == "__main__":
    main()
```

Рисунок 5 — CLI-оболонка програми та параметри запуску.

5. Створено тести pytest для перевірки коректності формування Top-N, сортування, відсутності мутації вхідних даних, правильності reduce-агрегації та роботи композиції функцій.



```python
import copy

from lab2 import build_pipeline, process_accounts, reduce_stats, with_turnover


def sample():
    return [
        {"id": 3, "name": "iryna melnyk", "city": "Kyiv", "transactions": [100, 200]},
        {"id": 1, "name": "anna kovalenko", "city": "Lviv", "transactions": [300]},
        {"id": 2, "name": "bohdan shevchenko", "city": "Kyiv", "transactions": [100, 100]},
    ]


def test_top_n_and_sorting():
    result = process_accounts(sample(), top_n=2, city="Kyiv", factor=1.1)
    assert [account["id"] for account in result] == [3, 1]
    assert result[0]["turnover"] == 330


def test_input_is_not_mutated():
    accounts = sample()
    original = copy.deepcopy(accounts)
    process_accounts(accounts, top_n=3, city="Kyiv", factor=1.1)
    assert accounts == original

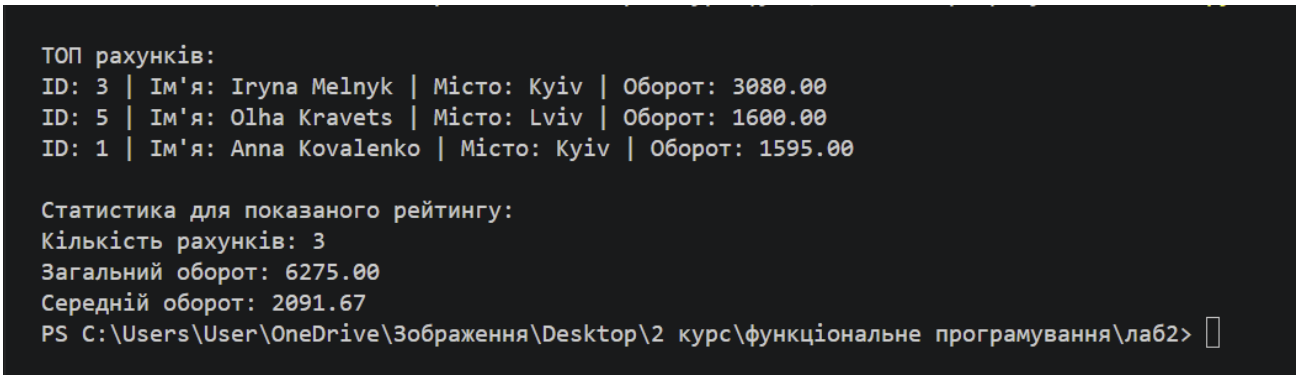

def test_reduce_stats():
    stats = reduce_stats([
        {"id": 1, "turnover": 10},
        {"id": 2, "turnover": 20},
    ])
    assert stats == {"count": 2.0, "turnover": 30.0, "avg_turnover": 15.0}


def test_empty_reduce_stats():
    assert reduce_stats([]) == {"count": 0.0, "turnover": 0.0, "avg_turnover": 0.0}


def test_pipeline_composition():
    result = build_pipeline(top_n=1, city="Kyiv", factor=1.0)(sample())
    assert len(result) == 1
    assert result[0]["id"] == 1
```

Рисунок 6 — Автоматичні тести лабораторної роботи.

Виконано тестування програми за допомогою команди python -m pytest -q. Тести перевіряють очікуваний порядок рахунків у рейтингу, правильність розрахунку обороту та статистики, збереження початкових даних без змін і коректну роботу конвеєра.



```
ТОП рахунків:
ID: 3 | Ім'я: Iryna Melnyk | Місто: Kyiv | Оборот: 3080.00
ID: 5 | Ім'я: Olha Kravets | Місто: Lviv | Оборот: 1600.00
ID: 1 | Ім'я: Anna Kovalenko | Місто: Kyiv | Оборот: 1595.00

Статистика для показаного рейтингу:
Кількість рахунків: 3
Загальний оборот: 6275.00
Середній оборот: 2091.67
PS C:\Users\User\OneDrive\Зображення\Desktop\2 курс\функціональне програмування\лаб2>
```

Рисунок 7 — Результат запуску програми та виконання автоматичних тестів.

Висновок: У ході лабораторної роботи було реалізовано програму для аналізу банківських рахунків відповідно до 8-го варіанта. Було використано функції вищого порядку, lambda-вирази, генераторні вирази, композицію функцій та reduce-агрегацію. Програма обчислює оборот рахунків, застосовує коефіцієнт для вибраного міста, фільтрує записи за мінімальним оборотом, формує рейтинг Top-N і розраховує статистичні показники. Обробку даних відокремлено від введення-виведення, а початкові записи не змінюються. Коректність роботи перевірено за допомогою автоматичних тестів pytest.